

Modelling Data and Advanced Mongoose

Locations (Geospatial Data)

Sy: 1

```
6  const tourSchema = new mongoose.Schema(  
7    {  
92    startLocation: {  
93      // GeoJSON  
94      type: {  
95        type: String,  
96        default: 'Point',  
97        enum: ['Point'],  
98      },  
99      coordinates: [Number],  
100     address: String,  
101     description: String,  
102   },  
103   locations: [  
104     {  
105       type: {  
106         type: String,  
107         default: 'Point',  
108         enum: ['Point'],  
109       },  
110       coordinates: [Number],  
111       address: String,  
112       description: String,  
113       day: Number,  
114     },  
115   ],  
116   guides: [  
117     {  
118       type: mongoose.Schema.ObjectId,  
119       ref: 'User',  
120     },  
121   ],  
122 },  
127 );
```

Start Location için kullanılan **Geojson** veri tipi.

Locations dizisi için de **Geojson** veri tipi kullanılmakta.

Modelling Data and Advanced Mongoose

Populating 1

Sy: 2

tourSchema.guides Property

```
6 const tourSchema = new mongoose.Schema({
7   {
8     // ...
9     // ...
10    // ...
11    guides: [
12      {
13        type: mongoose.Schema.ObjectId,
14        ref: 'User',
15      },
16    ],
17  },
18 });
```

tourSchema.find populate

```
1 tourSchema.pre(/^find/, function (next) {
2   this.populate({
3     path: 'guides',
4     select: '-__v -passwordChangedAt',
5   });
6   next();
7 });
```

Get Tour

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c2951

Params Authorization Headers (9) Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies (1) Headers (21) Test Results Visualize

JSON Preview Visualize

```
26 {
27   "secretTour": false,
28   "guides": [
29     {
30       "role": "guide",
31       "_id": "682cb8aaad40925ec430c628",
32       "name": "User",
33       "email": "guide@jonas.io"
34     }
35   ],
36   "_id": "5c88fa8cf4afda39709c2951",
37   "name": "The Forest Hiker",
38   "duration": 5,
39   "maxGroupSize": 25,
40   "difficulty": "easy",
41   "price": 397,
```

Tour şeması üzerinde oluşturulan **guides property'si** **User** şeması ile ilişki kurmak için kullanılan bir alandır. Tour şeması üzerinde bulunan **guides** özelliği turda görevli kullanıcıların **id** bilgilerini tutar. Guides ve id üzerinden kurulan ilişki ile turda görevli kullanıcı bilgileri alınabilir. Tour bilgileri üzerinde yapılan her **find** ile başlayan sorgularda Tour.guides üzerinde tutulan idler üzerinden **this.populate()** metodu işletilerek kullanıcıların diğer bilgileri getirilir.

Modelling Data and Advanced Mongoose

Review Schema

Sy: 3

reviewSchema

```
1 const reviewSchema = new mongoose.Schema(
2   {
3     review: {
4       type: String,
5       required: [true, 'Review can not be empty!'],
6     },
7     rating: {
8       type: Number,
9       min: 1,
10      max: 5,
11    },
12    createdAt: {
13      type: Date,
14      default: Date.now,
15    },
16    tour: {
17      type: mongoose.Schema.ObjectId,
18      ref: 'Tour',
19      required: [true, 'Review must belong to a tour.'],
20    },
21    user: {
22      type: mongoose.Schema.ObjectId,
23      ref: 'User',
24      required: [true, 'Review must belong to a user'],
25    },
26  },
27  {
28    toJSON: { virtuals: true },
29    toObject: { virtuals: true },
30  },
31 );
32
33 reviewSchema.pre(/^find/, function (next) {
34   this.populate({
35     path: 'tour',
36     select: 'name',
37   }).populate({
38     path: 'user',
39     select: 'name photo',
40   });
41   next();
42 });
43
```

reviewController

```
1 exports.getAllReviews = catchAsync(async (req, res, next) => {
2   const reviews = await Review.find();
3
4   res.status(200).json({
5     status: 'success',
6     results: this.getAllReviews.length,
7     data: {
8       reviews,
9     },
10  });
11 });
12
13 exports.createReview = catchAsync(async (req, res, next) => {
14   const newReview = await Review.create(req.body);
15
16   res.status(201).json({
17     status: 'success',
18     data: {
19       review: newReview,
20     },
21  });
22 });
```

Create New Review

HTTP Jonas Natours 2025 / Review / Create New Review

POST {{URL}} api/v1/reviews

Params Authorization Headers (11) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL

```
1 {
2   "review": "Loved it!",
3   "rating": 4,
4   "tour": "5c88fa8cf4afda39709c2951",
5   "user": "682da32af55f7ce393e30f41"
6 }
```

Body Cookies (1) Headers (21) Test Results

JSON Preview Visualize

```
1 {
2   "status": "success",
3   "data": {
4     "review": {
5       "_id": "682da3baf55f7ce393e30f47",
6       "review": "Loved it!",
7       "rating": 4,
8       "tour": "5c88fa8cf4afda39709c2951",
9       "user": "682da32af55f7ce393e30f41",
10      "createdAt": "2025-05-21T09:58:18.510Z",
11      "__v": 0,
12      "id": "682da3baf55f7ce393e30f47"
13    }
14  }
```

Get Review

GET {{URL}} api/v1/reviews

Params Authorization Headers (9) Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies (1) Headers (21) Test Results

JSON Preview Visualize

```
31 {
32   "_id": "682da3baf55f7ce393e30f47",
33   "review": "Loved it!",
34   "tour": {
35     "guides": [
36       {
37         "role": "guide",
38         "_id": "682cb8aaad40925ec430c628",
39         "name": "User",
40         "email": "guide@jonas.io"
41       }
42     ],
43     "_id": "5c88fa8cf4afda39709c2951",
44     "name": "The Forest Hiker",
45     "durationWeeks": null,
46     "id": "5c88fa8cf4afda39709c2951"
47   },
48   "user": {
49     "_id": "682da32af55f7ce393e30f41",
50     "name": "User"
51   },
52   "createdAt": "2025-05-21T09:58:18.510Z",
53   "__v": 0,
54   "id": "682da3baf55f7ce393e30f47"
55 }
```

Turlar hakkında yapılan görüşleri **Review Schema** üzerinden veritabanına yazılır. Girilen bir **review** üzerinde **tourId** ve **userId** bulundurmak tour ve user dökümanlarının gereksiz büyümesini engeller. Gerektiğinde review üzerinde bulunan tourId ve userId ile ilgili dökümanların bilgileri **populate** metoduyla çekilebilir.

Modelling Data and Advanced Mongoose

Populate Fault!

Sy: 4

Get A Tour

```
{
  "imageCover": "tour-1-cover.jpg",
  "locations": [
    {
      "lat": 34.04,
      "long": 15.03
    }
  ],
  "slug": "the-forest-hiker",
  "durationWeeks": 0.7142857142857143,
  "reviews": [
    {
      "_id": "682da3baf55f7ce393e30f47",
      "review": "Loved it!",
      "tour": {
        "_id": "5c88fa8cf4afda39709c2951",
        "name": "The Forest Hiker",
        "durationWeeks": null,
        "id": "5c88fa8cf4afda39709c2951"
      },
      "user": {
        "_id": "682da32af55f7ce393e30f41",
        "name": "User"
      },
      "createdAt": "2025-05-21T09:58:18.510Z"
    }
  ]
}
```

reviewSchema Find Middleware

```
33 reviewSchema.pre(/^find/, function (next) {
34   this.populate({
35     path: 'tour',
36     select: 'name',
37   }).populate({
38     path: 'user',
39     select: 'name photo',
40   });
41   next();
42 });
```

reviewSchema Middleware Fix

```
1 reviewSchema.pre(/^find/, function (next) {
2   this.populate({
3     path: 'user',
4     select: 'name photo',
5   });
6   next();
7 });
```

```
{
  "imageCover": "tour-1-cover.jpg",
  "locations": [
    {
      "lat": 34.04,
      "long": 15.03
    }
  ],
  "slug": "the-forest-hiker",
  "durationWeeks": 0.7142857142857143,
  "reviews": [
    {
      "_id": "682da3baf55f7ce393e30f47",
      "review": "Loved it!",
      "tour": {
        "_id": "5c88fa8cf4afda39709c2951",
        "name": "The Forest Hiker",
        "durationWeeks": null,
        "id": "5c88fa8cf4afda39709c2951"
      },
      "user": {
        "_id": "682da32af55f7ce393e30f41",
        "name": "User"
      },
      "createdAt": "2025-05-21T09:58:18.510Z"
    }
  ]
}
```

tourSchema Middleware Fix (Virtual Property)

```
1 // Virtual populate
2 tourSchema.virtual('reviews', {
3   ref: 'Review',
4   foreignField: 'tour',
5   localField: '_id',
6 });
```

tourControl.getTour()

```
1 exports.getTour = catchAsync(async (req, res, next) => {
2   const tour = await Tour.findById(req.params.id).populate('reviews');
3
4   if (!tour) {
5     return next(new AppError('No tour found with that ID', 404));
6   }
7
8   res.status(200).json({
9     status: 'success',
10    data: {
11      tour,
12    },
13  });
14 });
```

Review Schema üzerinde **tour** ve **user** için populate işlemi uygulanarak bir **tour** dokümanı çekildiğinde, **tour** içinde bulunan **review** bilgisi üzerinden yeniden **tour** bilgisi gelir. Bilginin bu şekilde sürekli tekrar etmesi istenen bir durum değildir. Bu durumu düzeltmek için **reviewSchema** üzerinde bulunan **find middleware** üzerinde ki **tour populate** silinir. **tourSchema** üzerinde gereksiz alan barındırmamak için **review** dökümanına ait bir bilgi saklanmaz. Fakat **tour** üzerinde **review** bilgisini görüntülemek gerektiğinde bu durum çözmek için **virtual property** tanımlanabilir. **Virtual Property** hafızada çalışan sanal bir alan olduğu için bilgisayarda yer kaplamaz. **tourSchema** üzerinde **reviews** için **virtual property** hazırlandıktan sonra, **tourControl.getTour()** metodunda **populate('reviews')** uygulanır.

Modelling Data and Advanced Mongoose

Express Nested Routes

Sy: 6

Tour Routes

```
routes > JS tourRoutes.js > ...
27   authController.protect,
28   authController.restrictTo('admin', 'lead-guide'),
29   tourController.deleteTour,
30   );
31
32   // POST /tour/234fad4/reviews
33   // GET /tour/234fad4/reviews
34   // GET /tour/234fad4/reviews/94887fda
35
36   router
37     .route('/:tourId/reviews')
38     .post(
39       authController.protect,
40       authController.restrictTo('user'),
41       reviewController.createReview,
42     );
43
44   module.exports = router;
```

Review Routes

```
routes > JS reviewRoutes.js > ...
You, 23 hours ago | 1 author (You)
1   const express = require('express');
2   const reviewController = require('../controllers/reviewController');
3   const authController = require('../controllers/authController');
4
5   const router = express.Router();
6
7   router
8     .route('/')
9     .get(reviewController.getAllReviews)
10    .post(
11      authController.protect,
12      authController.restrictTo('user'),
13      reviewController.createReview,
14    );
15
16   module.exports = router;
```

Tour Routes Fix

```
1   const express = require('express');
2   const reviewRouter = require('../routes/reviewRoutes');
3
4   const router = express.Router();
5
6   router.use('/:tourId/reviews', reviewRouter);
7
8   module.exports = router;
```

Review Routes Fix

```
1   const express = require('express');
2   const reviewController = require('../controllers/reviewController');
3   const authController = require('../controllers/authController');
4
5   const router = express.Router({ mergeParams: true });
6
7   router
8     .route('/')
9     .get(reviewController.getAllReviews)
10    .post(
11      authController.protect,
12      authController.restrictTo('user'),
13      reviewController.createReview,
14    );
15
16   module.exports = router;
```

Create a review on tour

POST (URL) api/v1/tours/5c88fa8cf4afda39709c295d/reviews

Send

```
1 {
2   "rating": 3,
3   "review": "Was kind okay"
4 }
```

Body

201 Created - 19 ms - 1.19 KB

JSON

```
1 {
2   "status": "success",
3   "data": {
4     "review": {
5       "_id": "682e21ba6978d361a48525c6",
6       "rating": 3,
7       "review": "Was kind okay",
8       "tour": "5c88fa8cf4afda39709c295d",
9       "user": "682e168b51add1587411ee3e",
10      "createdAt": "2025-05-21T18:55:54.645Z",
11      "__v": 0,
12      "id": "682e21ba6978d361a48525c6"
13    }
14  }
15 }
```

Get tour

GET (URL) api/v1/tours/5c88fa8cf4afda39709c295d

Send

```
60   "coordinates": [
61     -122.408865,
62     37.787825
63   ],
64   "_id": "5c88fa8cf4afda39709c295e",
65   "description": "San Francisco",
66   "day": 5
67 },
68 ],
69 "slug": "the-city-wanderer",
70 "__v": 0,
71 "durationWeeks": 1.2857142857142858,
72 "reviews": [
73   {
74     "_id": "682e21ba6978d361a48525c6",
75     "rating": 3,
76     "review": "Was kind okay",
77     "tour": "5c88fa8cf4afda39709c295d",
78     "user": {
79       "_id": "682e168b51add1587411ee3e",
80       "name": "jonas"
81     }
82   }
83 ]
```

En üstte bulunan simple nested routes çözümü **tourRoutes** ve **reviewRoutes** üzerinde kod tekrarı oluşturur. Bu istenen bir durum değildir. Bu durumun çözümü için **router.use()** metodu kullanılır. **tourRoutes** üzerinde bulunan **router.use()** metodunun ilk parametresi route adresi, ikinci parametresi kullanılacak router metodudur(**reviewRouter**). **reviewRoutes** üzerinde **router** nesnesi oluşturulurken **express.Router()** metodu **mergeParams:true** değerini alarak router birleşimini gerçekleştirir.

Modelling Data and Advanced Mongoose

Create Reviews on Tour

Sy: 7

Post Review on Tour

```
1 //POST: {{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews
2 {
3   "rating": 4,
4   "review": "very good tour"
5 }
```

tourRoute

```
1 const express = require('express');
2 const reviewRouter = require('../routes/reviewRouter');
3
4 const router = express.Router();
5
6 router.use('/:tourId/reviews', reviewRouter);
7
8 module.exports = router;
```

reviewRoute

```
1 const express = require('express');
2 const reviewController = require('../controllers/reviewController');
3 const authController = require('../controllers/authController');
4
5 const router = express.Router({ mergeParams: true });
6
7 // POST /tour/234fad4/reviews
8 // GET /tour/234fad4/reviews
9 // POST /reviews
10
11 router
12   .route('/')
13   .get(reviewController.getAllReviews)
14   .post(
15     authController.protect,
16     authController.restrictTo('user'),
17     reviewController.createReview,
18   );
19
20 module.exports = router;
```

reviewController.createReview()

```
1 exports.createReview = catchAsync(async (req, res, next) => {
2   // Allow nested routers
3   if (!req.body.tour) req.body.tour = req.params.tourId;
4   if (!req.body.user) req.body.user = req.user.id;
5
6   const newReview = await Review.create(req.body);
7
8   res.status(201).json({
9     status: 'success',
10    data: {
11      review: newReview,
12    },
13  });
14 });
```

MongoDB

```
_id: ObjectId('68315308bfd2a56e2adbc387')
rating: 4
review: "very good tour"
tour: ObjectId('5c88fa8cf4afda39709c295d')
user: ObjectId('68315303bfd2a56e2adbc384')
createdAt: 2025-05-24T05:03:04.526+00:00
```

tours route'u üzerinden gönderilen reviews post'u öncelikle tourRoute üzerinden yakalanır ve burada bulunan router.use() metodu ile bu talep reviewRoute'una iletilir. tourRoute üzerinden gönderilen parametreler mergeParams:true parametresi ile birleştirilerek tourId değeri reviewRoute üzerinden alınır. reviewRoute üzerinden post işlemi öncelikle protect() metoduyla başlar. protect() metodu kullanıcının login olup olmadığına bakar. Eğer kullanıcı login olduysa user nesnesi protect() metodu üzerinden createReview() metoduna gönderilir.

createReview() metodu tourRoute() metodu üzerinden gelen parametre verisindeki tourId ve protect() metodundan gelen userId verisini alarak kullanıcı review bilgisini veritabanına yazar.

Modelling Data and Advanced Mongoose

Get Reviews on Tour

Sy: 8

{{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c295d/reviews

Send

tourRoute

```
1 const express = require('express');
2 const reviewRouter = require('../routes/reviewRoute');
3
4 const router = express.Router();
5
6 router.use('/:tourId/reviews', reviewRouter);
7
8 module.exports = router;
```

reviewRoute

```
1 const express = require('express');
2 const reviewController = require('../controllers/reviewController');
3 const authController = require('../controllers/authController');
4
5 const router = express.Router({ mergeParams: true });
6
7 // POST /tour/234fad4/reviews
8 // GET /tour/234fad4/reviews
9 // POST /reviews
10
11 router
12   .route('/')
13   .get(reviewController.getAllReviews)
14   .post(
15     authController.protect,
16     authController.restrictTo('user'),
17     reviewController.createReview,
18   );
19
20 module.exports = router;
```

reviewController.getAllReviews()

```
1 exports.getAllReviews = catchAsync(async (req, res, next) => {
2   let filter = {};
3   if (req.params.tourId) filter = { tour: req.params.tourId };
4
5   // { tour: '5c88fa8cf4afda39709c295d' }
6
7   const reviews = await Review.find(filter);
8
9   res.status(200).json({
10     status: 'success',
11     results: reviews.length,
12     data: {
13       reviews,
14     },
15   });
16 });
```

reviewSchema

```
1 const reviewSchema = new mongoose.Schema(
2   {
3     review: {
4       type: String,
5       required: [true, 'Review can not be empty!'],
6     },
7     rating: {
8       type: Number,
9       min: 1,
10      max: 5,
11    },
12    tour: {
13      type: mongoose.Schema.ObjectId,
14      ref: 'Tour',
15      required: [true, 'Review must belong to a tour.'],
16    },
17    user: {
18      type: mongoose.Schema.ObjectId,
19      ref: 'User',
20      required: [true, 'Review must belong to a user'],
21    },
22  },
23  {
24    timestamps: true,
25  }
26 );
```

GET {{URL}} api/v1/tours/5c88fa8cf4afda39709c295d/reviews

Params Auth Headers (9) Body Scripts Settings Cookies

Body 200 OK 34 ms 1.46 KB Save Response

JSON Preview Visualize

```
{
  "status": "success",
  "results": 2,
  "reviews": [
    {
      "_id": "682e251ec3ecb67e7a17f1a6",
      "rating": 3,
      "review": "Was kind okay",
      "tour": "5c88fa8cf4afda39709c295d",
      "user": {
        "_id": "682da32af55f7ce393e30f41",
        "name": "User"
      }
    },
    {
      "_id": "68315308bfd2a56e2adbc387",
      "rating": 4,
      "review": "very good tour",
      "tour": "5c88fa8cf4afda39709c295d",
      "user": {
        "_id": "68315303bfd2a56e2adbc384",
        "name": "tuna"
      }
    }
  ]
}
```

get() metoduna gelene kadar post() metodunda bulunan adımlar aynı şekildedir.

tourRoute üzerinden gelen tourId Review schema'sı üzerinden sorgu yapabilmek için filter isimli nesneye dönüştürülür ve find() metoduna argüman olarak gönderilir.

Bu işlemin sonucu olarak bir tour'a bağlı olan review bilgileri nested route yardımıyla yapılır.

Modelling Data and Advanced Mongoose

Handler Factory Function

Sy: 9

Önceki Delete Tour Fonksiyonu

```
1 exports.deleteTourOld = catchAsync(async (req, res, next) => {
2   const tour = await Tour.findByIdAndDelete(req.params.id);
3
4   if (!tour) {
5     return next(new AppError('No tour found with that ID', 404));
6   }
7
8   res.status(204).json({
9     status: 'success',
10    data: null,
11  });
12 });
```

```
1 // Handler Factory Function
2 exports.deleteTour = factory.deleteOne(Tour);
```

deleteOne()
Generic
Function

```
1 exports.deleteOne = (Model) =>
2   catchAsync(async (req, res, next) => {
3     const doc = await Model.findByIdAndDelete(req.params.id);
4
5     if (!doc) {
6       return next(new AppError('No document found with that ID', 404));
7     }
8
9     res.status(204).json({
10      status: 'success',
11      data: null,
12    });
13  });
```

Benzer kullanım alanına sahip control fonksiyonlarını her veri için farklı şekilde tanımlamak ve kullanmak yerine **Generic Fonksiyonları** (Handler Factory Function) kullanmak çok daha kullanışlıdır. Bir Tour dökümanını silmek için kullanılan deleteTour() fonksiyonu yerine model tipini argüman olarak alan ve bu argüman üzerinden işlemi yapan bir fonksiyonu döndüren (**Closures**) generic bir fonksiyon tanımlamak daha kullanışlıdır. Üstte tanımlanan deleteOne(Model) generic fonksiyonu sadece Tour verisi için değil User, Review... gibi bir çok model için kullanılabilir.

```
1 exports.getOne = (Model, popOptions) =>
2   catchAsync(async (req, res, next) => {
3     let query = Model.findById(req.params.id);
4     if (popOptions) query = query.populate(popOptions);
5     const doc = await query;
6
7     if (!doc) {
8       return next(new AppError('No document found with that ID', 404));
9     }
10
11     res.status(200).json({
12       status: 'success',
13       data: {
14         data: doc,
15       },
16     });
17   });
```

```
1 exports.getUser = factory.getOne(User);
```

getOne(Model, popOptions)

```
1 exports.getTour = factory.getOne(Tour, { path: 'reviews' });
```

getOne() fonksiyonunun kullanımı incelendiğinde hem bir **User** dökümanı hem de bir **Tour** dökümanı almak için kullanılabildiği görülmekte. Eğer bu fonksiyonun ile alınan dökümanın kendi içinde bağlantılı bulunan başka döküman bilgisini de almak isteniyorsa, parametre olarak populate edilecek modelin bilgisi argüman olarak **getOne()** fonksiyonuna verilebilir.

```
1 router.get(
2   '/me',
3   authController.protect,
4   userController.getMe,
5   userController.getUser,
6 );
```

{{URL}}api/v1/users/me

```
1 exports.protect = catchAsync(async (req, res, next) => {
2   // *** The code exists above.
3   // GRANT ACCESS TO PROTECTED ROUTE
4   req.user = currentUser;
5   next();
6 });
```

```
1 exports.getMe = (req, res, next) => {
2   req.params.id = req.user.id;
3   next();
4 };
```

```
1 exports.getUser = factory.getOne(User);
```

```
1 exports.getOne = (Model, popOptions) =>
2   catchAsync(async (req, res, next) => {
3     let query = Model.findById(req.params.id);
4     if (popOptions) query = query.populate(popOptions);
5     const doc = await query;
6
7     if (!doc) {
8       return next(new AppError('No document found with that ID', 404));
9     }
10
11     res.status(200).json({
12       status: 'success',
13       data: {
14         data: doc,
15       },
16     });
17   });
```

"users/me" route ile logın olmuş kullanıcının bilgileri alınır. protect() fonksiyonuyla kullanıcının logın olduğu kontrol edilir ve logın olan kullanıcı nesnesi **req.user** olarak **request**'e yüklenir. **getMe()** metoduyla **getOne()** fonksiyonun kullanacağı **req.params.id** parametresi hazırlanır ve **getOne()** fonksiyonuna yönlendirilir. Son olarak da **getOne()** ile ayrı bir fonksiyonun kullanılmadan **req.params.id** ile kullanıcı verisi veri tabanından alarak **response(res)** ile gönderilir.

Modelling Data and Advanced Mongoose

Handler Factory Function

Sy: 10

tourRoute

```
6 router.use('/:tourId/reviews', reviewRouter);
```

reviewRoute

```
5 const router = express.Router({ mergeParams: true });

1 router
2   .route('/')
3   .get(reviewController.getAllReviews)
4   .post(
5     authController.protect,
6     authController.restrictTo('user'),
7     reviewController.setTourUserIds,
8     reviewController.createReview,
9   );
```

Post Review on Tour

```
1 //POST: {{URL}}api/v1/tours/5c88fa8cf4afda39709c295d/reviews
2 {
3   "rating": 4,
4   "review": "very good tour"
5 }
```

tourId üzerinden bir **reviews** oluşturma **post request'i** öncelikle **tourRoute** üzerinden **reviewRouter'a** yönlendirilir. **reviewRouter** **mergeParams:true** ayarı ile kullanıcının gönderdiği parametreleri birleştirir ve **post request'ini** işletmeye başlar.

- **protect** middleware kullanıcı login kontrolü yapar ve login olmuş kullanıcı objesini **req(request'e)** yükler ve bir sonraki adıma **next** ile yollar.
- **restrictTo** middleware kullanıcı yetkilerini inceler. Sadece **user rolündeki** kullanıcı bir review tanımı oluşturabilir yetkisi tanımlandığından harici durumlar hata verdirilerek yetkisiz eylemler engellenir. Hata yoksa bir sonraki adıma geçilir.
- **setTourUserIds** middleware ile **tour router** üzerinden gelen parametre üzerinden gönderilen **tourId** ve **protect** middleware tarafından gönderilen **user.id** bilgisi alınır. Bir sonraki eylem olan **createReview()** metoduna geçilir.
- **createReview()** metodu en sonunda kullanıcı tarafından gönderilen review request'ini veri tabanına işler.

```
1 exports.protect = catchAsync(async (req, res, next) => {
2   // *** The code exists above.
3   // GRANT ACCESS TO PROTECTED ROUTE
4   req.user = currentUser;
5   next();
6 });
```

```
1 exports.restrictTo = (...roles) => {
2   return (req, res, next) => {
3     // roles ['admin', 'lead-guide']. role='user'
4     if (!roles.includes(req.user.role)) {
5       return next(
6         new AppError('You do not have permission to perform this action', 403),
7       );
8     }
9     next();
10  };
11 };
12
```

```
1 exports.setTourUserIds = (req, res, next) => {
2   // Allow nested routers
3   if (!req.body.tour) req.body.tour = req.params.tourId;
4   if (!req.body.user) req.body.user = req.user.id;
5   next();
6 };
```

```
1 exports.createReview = factory.createOne(Review);
```

```
1 exports.createOne = (Model) =>
2   catchAsync(async (req, res, next) => {
3     const doc = await Model.create(req.body);
4
5     res.status(201).json({
6       status: 'success',
7       data: {
8         data: doc,
9       },
10    });
11  });
```

createReview() fonksiyonu **createOne()** handler factory fonksiyonu tarafından oluşturulan kullanıcı review bilgisini veri tabanına yazan bir fonksiyondur.

createOne(Model) fonksiyonu argüman olarak aldığı **Model** sınıfına göre kullanıcı **req(request-istek)**'iğini veri tabanına yazacak fonksiyonu hazırlayıp döndüren javascript closures fonksiyonudur. **createOne()** fonksiyonu **User**, **Tour** gibi model sınıflarını da kullanarak bu modellere uygun fonksiyonları oluşturan kullanışlı jenerik bir fonksiyondur.